Universite Clermont Auvergne

Master PAR

Industrial Mobile Manipulation

Final Project Report

Students: LOUNAS Gana
OULD OULHADJ Reda
Supervisor: Mohammad ALKHATIB
Academic year: 2025–2026

Outline

Contents

Abstract

This project is about developping and simulating a mobile manipulator with ROS2 with the goal of following a technician and picking up tools off the ground in a workshop or a warehouse and handing over tools when needed. It combines a Franka Research 3 robotic arm with a parallel gripper mounted on a mobile base. The base is a BCR bot with differential drive and a 2D LiDAR for navigation. Nav2 is used for SLAM, obstacle avoidance and path planning. The human following feature uses detection using YOLOv11nano. Grasping and pick-and-place are done by fine-tuning a SmolVLA model with imitation learning on synthetic demonstrations collected in Isaac Sim. We built three simulation environments with increasing difficulty, from a simple FR3 table setup to a full mobile manipulator setup with floor pick and bin placement. The navigation and following stack was validated with a final goal error of 0.298 m and Kalman filtering reduced target jitter by around 55%. For manipulation, TinyVLA was not reliable in our experiments, while SmolVLA reached around 80% success in table setups and up to 60% in the mobile setup. The complete ROS2 pipeline is integrated and functional in simulation, and the main remaining bottleneck is contact grasp precision in highly randomized scenes.

Acknowledgments

List of Figures

List of Tables

Acronyms / Notation

ACT Action Chunking with Transformers.

AMCL Adaptive Monte Carlo Localization.

BCR BCR bot, differential-drive mobile base used in our platform.

DDPM Denoising Diffusion Probabilistic Model.

FR3 Franka Research 3 (7-DoF robot arm).

HDF5 Hierarchical Data Format version 5.

IK Inverse Kinematics

Nav2 Navigation 2, the ROS2 navigation stack used for global/local planning and goal execution.

ODE Ordinary Differential Equation.

SDE Stochastic Differential Equation.

SLAM Simultaneous Localization and Mapping for building a map and localizing in it.

TCP Tool Center Point.

VLA Vision-Language-Action policy family for multimodal robot behavior learning.

VLM Vision-Language Model.

YOLO You Only Look Once, real-time object detection model family used for detection.

Part I
Context and Objectives

1 Introduction

Industrial environments and in particular workshops have a lot of low productivity tasks. For example, walking and moving to get tools in order to perform different maintenance tasks. This can lead to delays in production and wasted time instead of performing the actual tasks. It opens the window to cobots that can help in reducing this strain and allowing the technicians to focus on what matters. For example, a report by Siemens in 2024 estimates that the 500 largest companies lose around 1.4 trillion dollars every year from unplanned downtime which is equal to around 11% of the whole revenue. For the automotive industry, the downtime is around 2.3 million dollars per hour [18]. Another report by SMRP (Society for maintenance and reliability professionals) shows us that "good traditional" maintenance organizations waste around 30% of time in wrench time. More than 5h of each shift is usually wasted by operations like waiting and searching for parts and tools instead of actually doing the work [19].

Figure 0.1: Illustration of the tool-search problem context in a workshop-like layout [5].

Industrial mobile manipulation has seen a big surge this decade. They aim to couple mobility with interaction (often with a robotic arm) in order to execute a fetch, pick, place and inspect work in spaces where full automation isn't possible. What makes these systems difficult is that the mobility adds a difficulty layer on top of manipulation adding pose uncertainty and increases the chances of obstruction. Mobile manipulators also require many fields : perception, navigation, tasks, path and grasp planning, control, error recovery, human robot interaction and robotic hardware development. Each one of these fields is an area of research in itself. But the real challenge is to navigate and perform the task around people and infrastructure [16].

The international federation of robotics (IFR) reports that the sales of service robots is up 30% in 2023 (205000 units sold). The transportation and logistics is the biggest application of mobile manipulators with around 113000 units sold in 2023 according to IFR [8].

The most common engineering pattern for mobile manipulators is a staged autonomy stack which decomposes the task into perception, state estimation, task level planning and decision making and motion planning, control, grasp strategy and execution monitoring. [16].

Manipulation tasks in warehouse or workshops environment require a strong generalization to occlusions, randomly placed objects, lighting differences, new objects to pick up and different reflectance of the objects. These environments present multiple challenges. Among them, the available camera views are often limited which makes it difficult to reconstruct an object model. Another challenge is the fact that we often have occlusions between objects. This makes it hard for robots to manipulate objects and even detect them in these environments. We can divide manipulation approaches into 3 types of methods: Analytical methods, learning based methods and foundation models based methods

Classical analytical methods (non-learning) control are grouped into interpolation-based planning (offline polynomial trajectories between predefined states), sampling-based planning (example: RRT/PRM families for feasible collision-free paths in high-dimensional spaces), and optimization-based planning, where offline optimizers improve trajectory quality but are less reactive, while online MPC replans in receding horizon to handle disturbances. Learning-based methods formulate manipulation as an MDP and mainly include reinforcement learning and imitation learning: RL maximizes expected return, while imitation learnng is divided into behavior cloning, inverse RL etc. Foundation-model-based methods use pretrained multimodal backbones and end-to-end action policies, including vision encoders (example : ViT), vision-language encoders (example: SigLIP), language backbones (LLMs), VLM backbones (LLaVA...), and VA/VLA policies. foundation VLMs are usually fine-tuned with robot trajectories to map visual observations and language instructions directly to robot actions [1].

2 Proposed Solution

In this project we aim to create an industrial mobile manipulator assistant which is a robot that operates in human workspaces and follows a technician, detects and retrieves tools, and places or delivers them on request. We validate the project in simulation (Gazebo and IsaacSim using ROS2) and we use vision language action models (we finetune SmolVLA) for manipulation and pick and place for our specific tasks.

The mobile cobot we aim to build is a mobile shadowing robot that can follow a technician, stay near the operators and when the operator or technician leaves or asks for a tool, the robot detects it or takes a look at its database (To see whether the tool is in the bin or in a designated place) then it'll go and retrieve it to hand it over. It will coexist with humans hence the "cobot" name in small to medium sized workshops. We think this platform can get more useful with scale to have a real assistant in different environments and tasks and not only for the specific tasks we are focusing in this study.

The robot will be able to navigate indoors (inside the workshop) and react to obstacles to avoid them. It should be able to detect tools and humans and track the human in order to follow them while keeping a safe distance (using the depth camera).

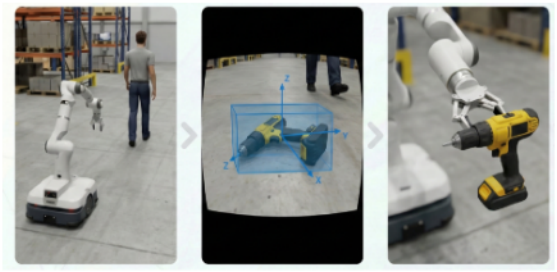


Figure 0.2: workflow: technician following, tool localization, and retrieval [5].

The manipulation tasks must be adaptive, meaning it should be robust to lighting changes, environment changes, different tools and be able to pick up objects in minor occlusions and cluttered environments. This is why we're using a VLA, which thanks to its pretraining scale and fine tuning in diverse demonstrations, can be less brittle than scripted policies (like the one we use for data generation)

In order to train the VLA, we need many diverse demonstrations. Collecting these demonstrations manually is slow and almost impossible in reality. We need a simulation environment with good domain randomization, different environments, different objects, occlusions and lighting. Then the data collection can be automated to generate multiple demonstrations.

Finally, the whole pipeline needs to be deployable in ROS with self-containing packages and nodes that take messages through topics as inputs and output commands directly to the robot.

3 Contributions

For this project we first studied the state of the art for both the main features we will be using on the mobile base and the VLAs for the manipulation part, then we selected the modules we judged to be adapted for our use case. we ended up choosing two VLA models: TinyVLA and SmolVLA.To gradually validate our choices we built three simulation environments : a simple FR3 table setup, a harder randomized table setup, and a mobile manipulator setup where the tool is picked from the floor and placed in a bin. We used both Gazebo and Isaac Sim. We deployed the mobile base stack and the VLA stack, and also built data-collection pipeline to acquire training data. After that we startred the training and evaluation of both selected models. The main practical outcome we got from all the experiences we've done is that SmolVLA gave more reliable results in our setup with same amount of data, so we kept it as the final baseline. This gives a clear failure/success analysis, and a roadmap toward building a deployable industrial assistant.

Part II
State of the Art and Bibliography

4 Vision-Language-Action Models for Robotics (Architectures, Training, Benchmarks)

With the rapid growth and advancements in LLMs (large language models) and vision language models (VLMs), Vision language action models are gaining a lot of popularity lately with the aim of creating generalist policies for manipulators and mobile platforms that work in different environment with minimal retraining by pretraining the models on massive datasets and benefiting from the massive scale that LLMs are trained with (Thousands of GPUs for several months, on internet scale data) which adds significant intelligence and depth to the models with the aim of understanding the environment and adapting to different scenarios and new tasks without having to retrain.

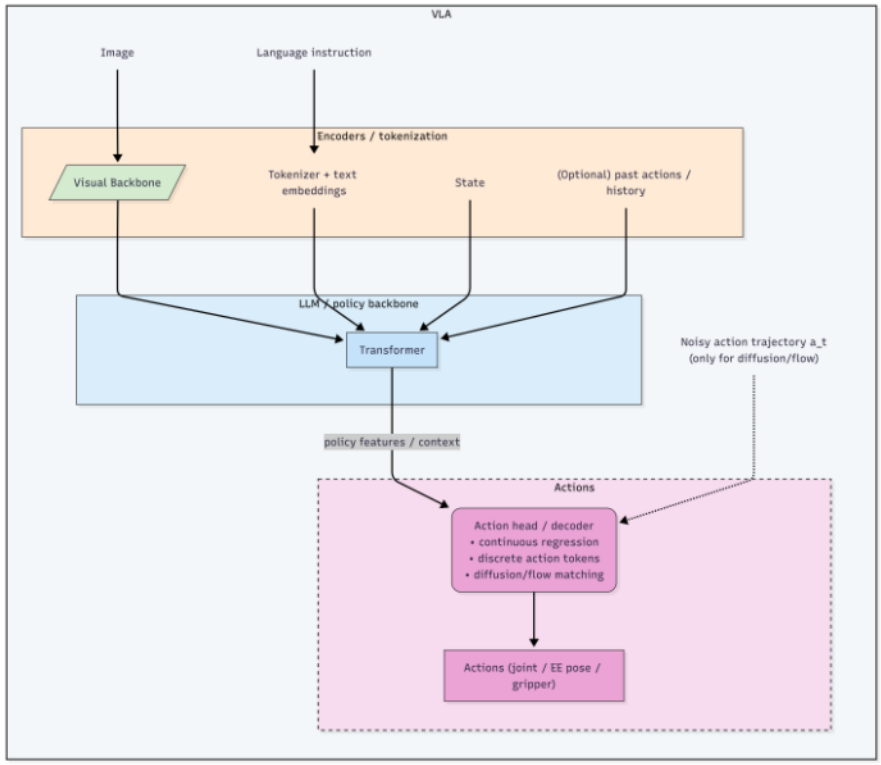


Figure 0.3: General architecture of a Vision Language Action model. based on [17], [2] and [12].

From the figure above, we can see the general architecture of some VLAs like smolVLA [17] and Pi0 [2] where an RGB or RGB-D image is encoded by a visual backbone into visual features. On the other side, the language instruction (eg: Pick up the screwdriver and put it in the bin) is tokenized into text embeddings. Additionally we provide the robot proprioception (state) and sometimes the past actions as well as additional inputs to the model. These are then combined into generally a multimodal transformer (as a backbone). Then additionally an action head maps the context from the transformer into robot controls (Future chunks (generally several steps ahead, sometimes only one action at a time)). The action heads come in different architectures from diffusion to flow matching policies, to continuous regression. Generally, we condition it on a noisy action trajectory during the iterative denoising.

Architecture

Generally, we have 7 different architectures for VLAs:

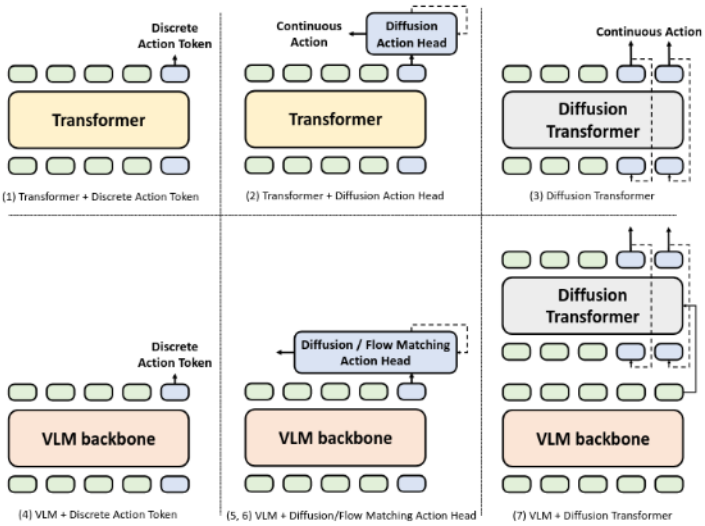


Figure 0.4: Overview of VLA architectures [10]. Top row: (1) Transformer with discrete action tokens, (2) Transformer with a diffusion action head, (3) Diffusion Transformer with continuous actions. Bottom row: (4) VLM backbone with discrete action tokens, (5, 6) VLM backbone with diffusion/flow matching action head, (7) VLM backbone with a Diffusion Transformer. Figure licensed under CC BY 4.0.

The 7 architectures can be grouped by backbone and action head [10]. The simplest designs use a plain transformer: a first variant discretizes continuous actions into tokens and predicts them either autoregressively (trained with cross-entropy) or in a single non-autoregressive forward pass. A second variant replaces the discrete head with a diffusion policy head that iteratively denoises a noisy future trajectory chunk conditioned on the transformer's multimodal context, to have smoother and more stable outputs. A third variant which is the diffusion transformer moves the denoising process inside the transformer which processes conditioning inputs, noisy action tokens and a diffusion timestep [10]. The remaining four architectures substitute the plain transformer with a VLM backbone pretrained on internet-scale data to improve generalization. Most modern VLAs follow this pattern [10]. On top of the VLM, the action head can be a discrete token decoder, a diffusion head connected to the VLM hidden states via cross-attention (queries are action tokens, keys and values are VLM tokens), a flow-matching head as in π_0 [2] which learns a continuous-time velocity field from noise to actions and requires fewer sampling steps, or a full diffusion transformer that acts as a low-level policy while the VLM serves as the high-level policy as in smolVLA [17, 10].

The table below shows the different architectures and benchmark values of VLAs.

Model	P(B)	Head	Rate	VRAM inf.	VRAM FT	Train scale	Score
RT-2	5/55	AR-tok	5 Hz (5B), 1–3 Hz (55B)	10+/110+	41e/456e	Web+robot co-FT	63
OpenVLA	7.2	AR-tok	∼6 Hz	15 (bf16), 7 (int4)	59.7 (LoRA-r32)	970k ep; 21.5k A100h	76.5 LBR
π_0	3.3	FM-cont	50 Hz; 73/86 ms	6.6+	∼27.4e	10k+ h; 903M ts	94.2 LBR
TinyVLA-H	1.3	Diff-cont	∼20× faster vs OpenVLA	2.6+	∼10.8e	50 demos/task (sim)	94.0 real
SmolVLA	0.45	FM-cont	Async: 19 vs 9 cycles	0.9+	∼3.7e	<30k ep; ∼30k GPUh	87.3 LBR
GR00T N1-2B	2.2	Hier-FM	S2:10 Hz, S1:120 Hz	4.4+	∼18.2e	∼50k H100h	76.8 real

Table 0.1: VLA comparison for one GPU, 3-view warehouse mobile manipulation. TinyVLA was used first for initial experiments, then SmolVLA was selected for the best compute/performance trade-off for a one GPU setup for training.

Abbreviations: AR-tok = autoregressive tokenized actions; FM-cont = flow-matching continuous head; Diff-cont = diffusion continuous head; Hier-FM = hierarchical multi-rate with flow/diffusion low-level control; LBR = LIBERO average. VRAM with "+" are lower bounds from parameter count ($\geq 2 \times P_B$ GB, bf16 weights only). "e" = estimation. LIBERO scores are not directly comparable between models.

Sources: RT-2 [3]; OpenVLA [11]; π_0 [2]; TinyVLA [21]; SmolVLA [17]; GR00T N1 [15].

VLA Choice and Architecture

From (Table 0.1), the two most adequate models for this project taking into consideration our computing limitations are TinyVLA and SmolVLA.

TinyVLA is in the **VLM + diffusion continuous action head** family.

TinyVLA uses a pretrained vision-language backbone as the multimodal context encoder then attaches a diffusion policy decoder that outputs *continuous* action chunks instead of autoregressive action tokens [21]. During robot adaptation, the architecture is tuned with parameter-efficient LoRA updates (instead of full-model retraining) while most backbone weights remain frozen. This reduces training cost and memory pressure while preserving the semantic priors of the VLM. The diffusion head is then trained to denoise noisy action trajectories conditioned on image, language and robot state which places TinyVLA in the "VLM + diffusion continuous head category. [21].

SmolVLA uses a similar design: a pretrained SmolVLM2 backbone produces visual-language conditioning tokens and a dedicated action expert (flow-matching transformer) predicts continuous action chunks [17]. The policy side is structured with interleaved *cross-attention* and *self-attention*: cross-attention injects VLM context (image, task text, and projected state), while self-attention models temporal dependencies inside the action chunk, starting from noisy actions and integrating toward executable controls. This places SmolVLA in the "VLM + flow-matching continuous head" category with lower sampling latency than diffusion-style denoising in deployment [17].

We kept TinyVLA (option A) and SmolVLA (option B) as the two practical architectures because both satisfy the same project constraint profile from Table 0.1: continuous-control outputs, substantially lower compute needed than very large VLAs and feasible one-GPU fine tuning. Methodologically, TinyVLA was the safer first step to validate data formatting, action representation and end-to-end training and inference wiring under limited resources while SmolVLA was the stronger final candidate for responsiveness and compute/performance balance [21, 17].

Flow Matching vs DDPM

Flow matching trains the action expert to predict the velocity vector $v_\theta(x_t, o_t, t)$ from the current noisy action chunk, the VLM features, and the flow time. At inference this velocity is integrated with an Euler ODE solver in a few steps.

Denoising diffusion probabilistic models (DDPM) train the network to predict the noise that was added to the clean data. During inference, we subtract iteratively the estimated noise along hundreds of steps using a stochastic differential equation (SDE).

Flow matching is a lot faster in inference and is simpler to train (MSE loss).

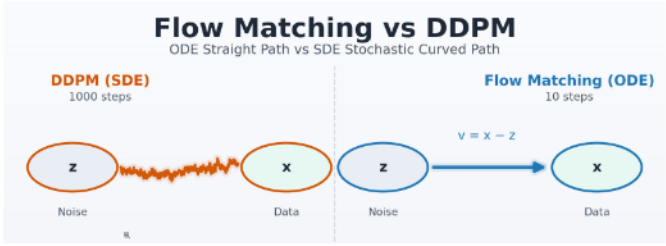


Figure 0.5: Flow Matching (straight ODE path, 10 steps, fast) vs DDPM (curved SDE path, 1000 steps, slow) [20]

Part III
Approach and Methodology

5 Use-Case Scenario: Technician Following, Tool Retrieval, and Bin Placement

This diagram shows the full mission logic and where each main block comes in. At the beginning, the robot follows the technician. In this phase, person detection and tracking are active from the front RGB-D stream, while **Nav2** keeps updating motion so the base stays behind the person and avoids obstacles with LiDAR safety rules.

When the robot reaches the work zone, it switches from follow mode to tool-search mode. At this point, **YOLO** is used for tool detection, and depth is used to estimate a usable 3D tool position. Then Nav2 is used again, now to drive the base to a pre-grasp area near the detected tool.

Once the base is in a valid pre-grasp pose, control priority moves to manipulation. This is where the **VLA policy** starts: it takes camera views, robot state, and task context, and outputs continuous arm and gripper actions for grasp and place. The recovery loop is also part of the use case: if detection fails, approach fails, or grasp fails, the system goes back to re-detection and re-approach, then retries under the same safety constraints.

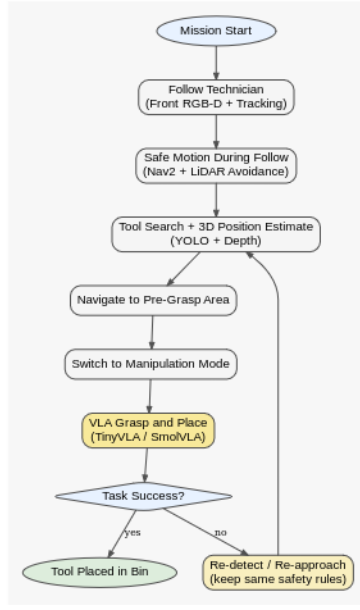


Figure 0.6: Use-case mission flow for technician following, tool retrieval, and bin placement.

6 Mobile System Architecture and Operation

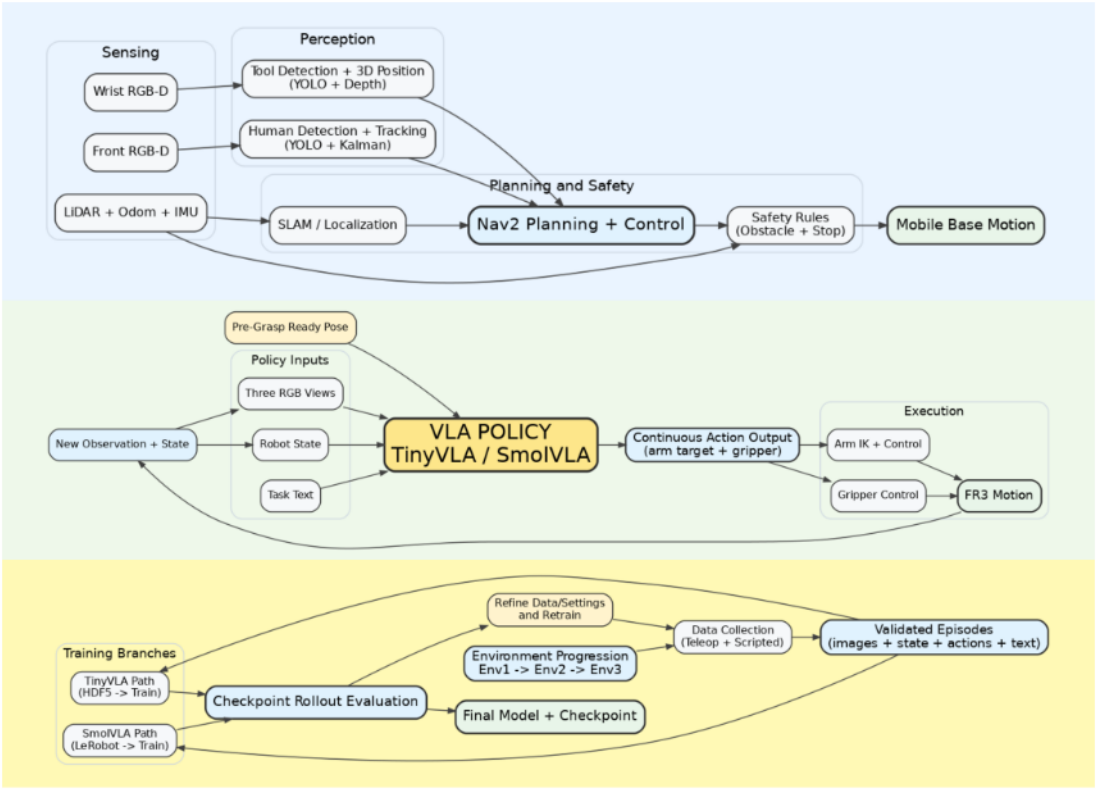


Figure 0.7: Top: mobile system architecture(Yolo for perception and nav2 for navigation); middle: manipulation and VLA architecture(observations as inputs and continious actions as outputs); bottom: data collection and training pipeline.

7 Human detection and tracking

Pipeline overview

A 3D person is tracked using the Kinect, we get an estimation of the position. We use YOLO11n-pose(Ultralytics, TensorRT deployment),and then a single person from multi-person detections is selected. Knee keypoints are used as the geometric anchor since the front camera used in here is mounted low on the base [9]. Those previously selected 2D keypoints are then fused with depth (median in a local window) and finlly projected to 3D.

This is all done in real time since we used a light yolo model, but there's a lot of noise because of occlusions and keypoint drops. Those cause as a side-effect a lot of jitter and unstability.

Temporal stabilization

To improve robustness, we track person position and velocity in 3D with a constant-velocity linear Kalman filter and update it with each valid RGB-D measurement. In addition to the standard predict/update cycle, we use two practical stabilizers: velocity decay after prediction (to reduce residual drift) and stationary snap when repeated measurements remain inside a small radius for multiple frames (to force near-zero velocity when the person is standing still).

All the details about the equations used and transition/observation matrices and covariance update are provided in Appendix 2 (*Methodology and Architecture - Additional Material*).

8 Tool Detection and Classification

The dection of tools is also run in real time ,for now we only tested it on the wrist camera. It outputs a class with a bounding box and a 3D target point in the camera frame.

Processing

Since the wrist camera is an RGB-D camera, we use Yolo on the RGB values and then synchronized it with the depth, then the highest confidence detection is chosen (the one highest above threshold). After that we calculate the center of the box and estimate depth of that point using local median window, and finally we project to 3D .

The same lightweight detector family used in this report (YOLO11 variants with TensorRT deployment) is kept here for consistent runtime constraints [9].

The Detailed equations and parameter used are provided in Appendix 2 (*Methodology and Architecture - Additional Material*).

9 Navigation and Human following

Tracking a human in 3D, the mobile base hast to allow two behaviors with one stack: person following and autonomous navigation for tool retrieval/deposition.

Architecture

Nav2 is used with SLAM Toolbox for the mapping and localization,and also for planning, control and smoothing of the commands [14]. As inputs there is LiDAR, odometry and TF. As an output there is `/cmd_vel`. While mapping phase the SLAM Toolbox is used to estimate `map`→`odom`.Then localization is performed on the saved map on the autonomous phase .

The follow node transforms the filtered human target into the map frame, computes a standoff goal, and sends a `NavigateToPose` command to Nav2.

Costmaps and SLAM

The global costmap is in general for long horizon route planning,while the local costmap is usually used for short-horizon obstacle handling. In our configuration, the global map uses of the static/obstacle/inflation layers while on the other hand the local map uses the inflation layers with regards to the LiDAR observations. [14].

The detailed formulations (unicycle kinematics, standoff goal equations and SLAM graph interpretation) are in Appendix 2 (*Methodology and Architecture - Additional Material*).

Remaining work: We could try adding actual vehicle dynamics (friction for exemple), and an MPC controller to improve jitter.

Architecture details moved to appendix

(Detailed ROS architecture is moved to Appendix E, *Methodology and Architecture - Additional Material.*)

10 Data Collection Methodology

We started from manual demonstrations for control and interface validation, then moved to automated collection methods.

Collection methods used: Firstly, we implemented keyboard control in order to control the end effector and gripper manually. We used this stage mainly for debugging. Then we moved to SpaceMouse control to collect expert demonstrations and finally we automated data collection to allow us to gather enough episodes for training.

Environment progression We also increased environment complexity starting from a **Simple table setup** with only the FR3 and a simple task. The second step uses the same table setup, but a bin was added behind and more complex objects and randomization. Finally, we moved to **a full mobile manipulator setup** with the robot on a mobile base along with a bin.

Control

For the action, we use a 6 DOF (3D position + 3D orientation) delta TCP (tool center point) pose in the robot's base frame as well as 1D gripper action.

This choice is to simplify the learning objective for the VLA model to focus on where to move the gripper instead of the joints. The IK controller uses DLS method which takes into account the singularities. On the low level, the joint controller is simulated with a high-gain PD controller.

Cameras Setup

We use a 3 camera setup in the environment at a resolution of 320×320.

- **Left and right cameras**: Mounted on the left and right sides of the robot workspace looking at the robot.
- **Wrist Camera**: Attached directly to the end effector.

Vision-Language-Action (VLA) Integration

Since the data is used to train smolVLA and TinyVLA models, in the logging, we add natural language instructions alongside the actions and observations.

Instruction Generation: The language prompts are dynamically added based on the active target object and the type of the task (for example, pick up the screwdriver)

Negative Examples (Language Mismatch): In order to avoid hallucination actions from the VLA when the requested object (from language prompt) is absent, we added a language mismatch probability that adds negative examples into the dataset (for example, language: "pick up the screwdriver", but on the scene there's only a cuboid). During these episodes, the expert policy doesn't pick anything which teaches the VLA to pair the language and the visual scene and output 0 commands in this case.

randomization, automated collection and logging in appendix

(domain randomization and data logging are in Appendix E, *Methodology and Architecture - Additional Material*.)

11 Environments

For data collection and training, Nvidia's IsaacSim is used. The choice is mainly due to its real time speed and visual fedelity (photo realistic environments) as well as the accurate physics engine (PhysX). ROS2 humble is used as the orchestrator and organizer.

Environment 1 (Table + FR3, Single Tool, Simple Pick-and-Place)

Environment 1 is our first manipulation benchmark, used to validate the complete learning loop in a controlled setup before moving to randomized scenes.
The platform is an FR3 manipulator on a table setup with a single orange screwdriver that spawns on the left side of the table with small variation (± 10 cm in x and y) and small yaw variation. Each episode follows the cycle: home pose → approach and grasp → move to right-side place pose → release → return to home pose.
As we can see in the data-collection video for this environment in folder `data_collection/env1/` [13], the demonstration follows exactly this cycle for each episode.

Figure 0.8: Environment 1: three-camera view (left, right, wrist) of the FR3 table-top setup with a single screwdriver pick-and-place task.

Environment 2 (Table + FR3, Multi-Object Randomized Scene)

During this environment, we randomized the size, colors and pose of the cuboids as well as the screwdrivers. The environment is still table top with FR3. The language task is "pick up the screwdriver" or "cube" with a probability of 0.3 for screwdrivers and 0.7 for cuboids in the dataset.
Environment and data collection videos are in `data_collection/env2/` [13].
Generalization attempt We ran a final experiment on the table top environment while adding more domain randomization.
The grey environment is replaced by a simulated photorealistic warehouse environment in Isaac-Sim. Initial pose and orientation of the gripper is randomized in a cube of edge length of 50cm. Difficulty presets are added with multiple objects on the table and we added language/visual mismatch.

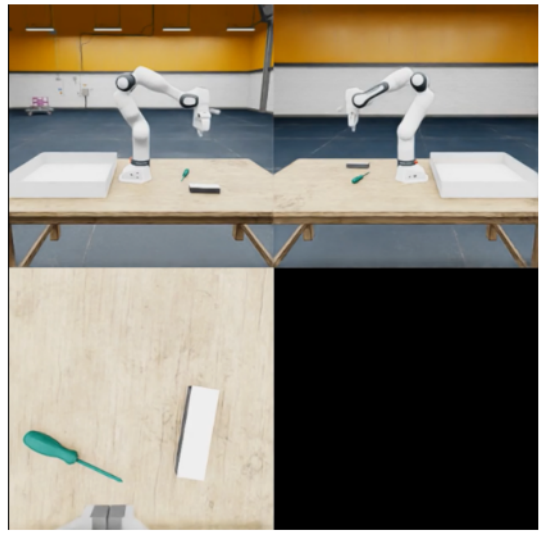


Figure 0.9: Environment 2: three-camera view (left, right, wrist) of the FR3 table-top setup with randomized multi-object scene.

Environment 3 (BCR + Mounted Manipulator, Floor Tool to Bin)

Environment 3 is the closest setup to our final use case. In this setup, the manipulator is mounted on the BCR mobile base, the screwdriver is spawned on the floor with strong randomization, and the task is to pick the screwdriver and place it in the bin. **Object**: one screwdriver with randomized floor pose. **Platform**: BCR mobile base and mounted manipulator. **Task**: pick up the screwdriver from the floor then place it in bin.
As we can see in folder `data_collection/env3/` [13], data collection for this setup is smoother than our first environment and closer to the intended deployment use case.

Figure 0.10: Environment 3: Base + manipulator, floor screwdriver pick-and-place to bin.

Part IV
Results and Analysis

12 Human detection, tracking and following results

Mapping performance

We mapped the environment using SLAM Toolbox by teleoperating the robot in the warehouse environment in Gazebo. We can see that the resulting occupancy grid in RViz correctly captures the walls, interior obstacles and open areas which makes it usable for our use case.

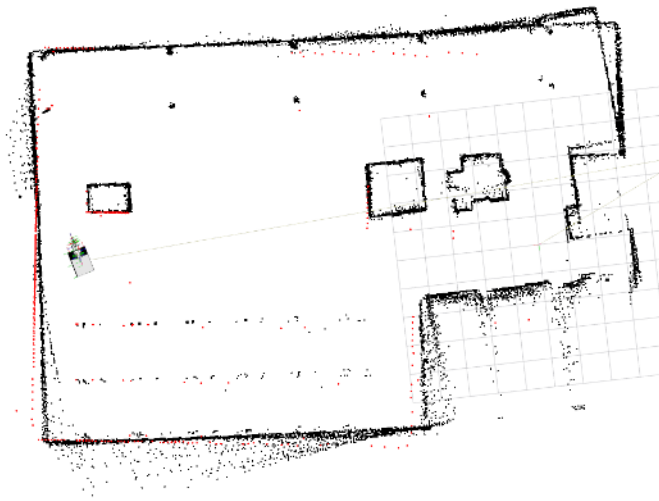


Figure 0.11: This figure shows in RViz the display of an occupancy grid map (made using slam tool box). The red dots are the current LiDAR scan synchronized with the map

To quantify localisation accuracy, the map-corrected odometry was compared against Gazebo ground truth in a full loop of the environment (tele-operated). Figure 0.12 overlays both trajectories. The two trajectories are aligned but we can see a divergence on the bottom right where the robot took

a long time to align the scans with the actual map. It's worth noting that all obstacles are correctly avoided and the trajectory becomes aligned again.

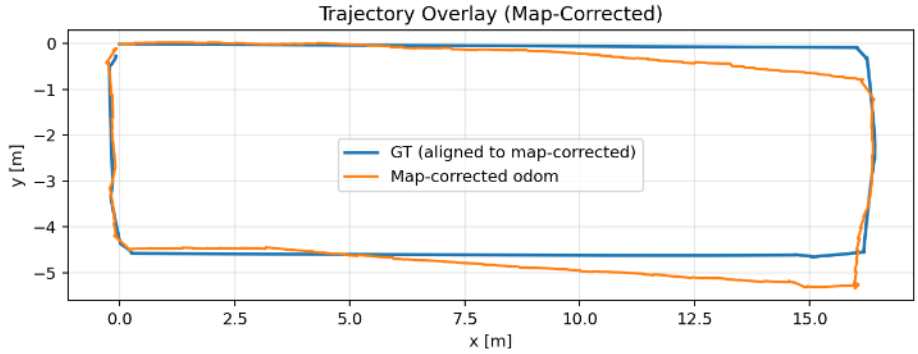


Figure 0.12: Trajectory overlay of the Gazebo ground-truth (blue) and the map-corrected odometry (orange) during a full loop of the environment.

Navigation performance

To evaluate the Nav2 navigation stack, the robot was commanded to reach a goal located around 15.6 m away (straight-line distance) in the mapped environment.

Table 0.2: Nav2 goal navigation metrics.

Metric	**Value**
Straight-line distance to goal	15.64 m
Trajectory length	23.30 m
Final goal error	0.298 m
Travel duration	89.2 s
Average speed	0.26 m/s

The robot reached the goal with a final positioning error of only 29.8 cm, which is well within the default Nav2 goal tolerance. The trajectory below shows that the map-corrected odometry and the AMCL localisation overlap during navigation, which confirms correct SLAM localisation.

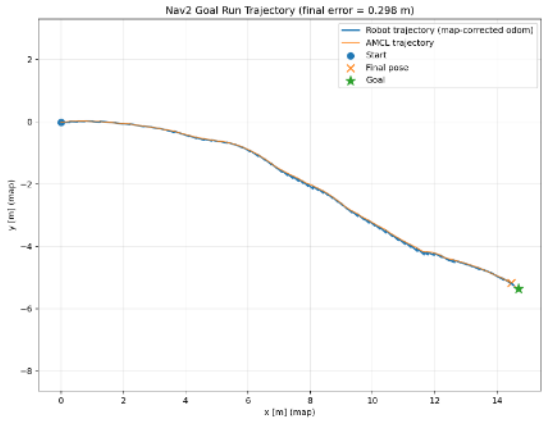


Figure 0.13: This represents a trajectory taken whilee navigating to a goal .The blue line is the corrected odometry and the orange one represents the AMCL estimation for the trajectory. The goal is the green star and the orange cross is the real final robot pose, we got a goal error of 0.298 m.

Human following and tracking

Using the kinect camera an estimation of the human is detected in frame camera. Figure 0.14 shows the frame where the knee keypoints (red dots) are used to estimate the 3D position of the human.

Figure 0.14: Pose estimation output from the simulated Kinect camera. The red dot indicates the detected knee keypoint used to derive the target's 3D position in the camera frame.

During human following, the robot kept a distance of around 1.63 m from the human on average, with an RMSE of 63 cm from the desired 1 m setpoint. This is mainly due to the acceleration/deceleration phases and the 180° turns of the human during the test as well as a conservative linear acceleration limit of 0.8 m/s^2 and a linear velocity limit of 0.45 m/s.

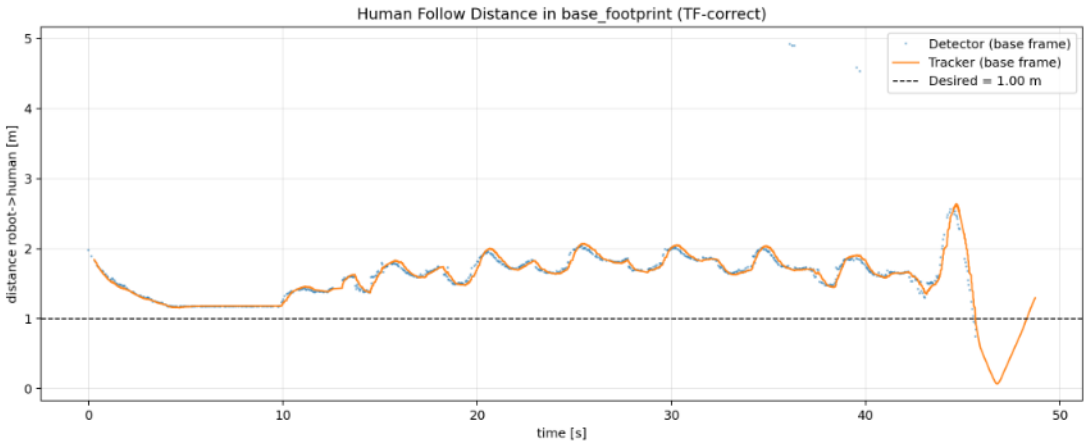
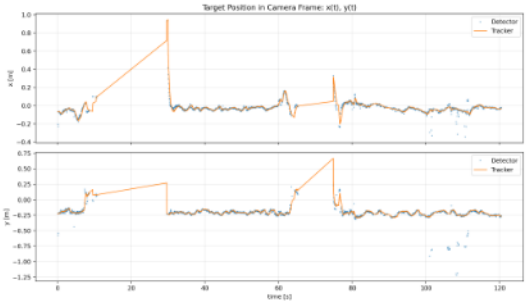


Figure 0.15: Human follow distance over time in the `base_footprint` frame. The dashed line indicates the desired following distance of 1 m. The detector raw measurements (blue dots) and the Kalman-filtered tracker output (orange line) both show the robot maintaining a distance of approximately 1.2 to 2 m, with occasional spikes during rapid human movements.

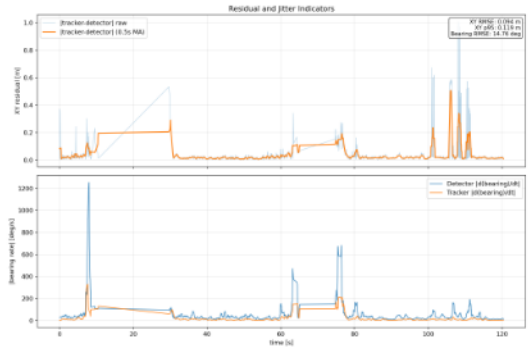
Kalman filter tracking performance

To evaluate the effect of the Kalman filter on tracking quality, the raw detector output and the filtered tracker output were compared over a 120 s run. The detector publishes at approximately 11.7 Hz, while the Kalman tracker interpolates this to 30 Hz.

In Figure 0.16a, we can see the target position in the camera frame over time. When we have detections, the tracker follows them correctly but when the human leaves the field of view, the tracker continues predicting using the constant velocity model which can cause drift until we get a new detection. In Figure 0.16b, we can see the tracker detector residual and jitter. The XY residual RMSE between the tracker and detector is 0.094 m, with a 95th-percentile of 0.119 m which confirms that the filter stays close to the measurements in normal conditions. Furthermore, the Kalman filter significantly reduces the jitter. The detector's bearing rate reaches a 95th-percentile of 137.9 °/s, whereas the tracker reduces this to 62.2 °/s which is a 55 % reduction which means it allows us to have a much smoother robot motion.



(a) We can see the first raw measurements detected (the blue dots)against the Kalman filtered output (the orange line). The imporved tracker is smoothly predictiong the detection even wen there are small occlusions, but it does drift for prolonged detectionn loss.



(b) Residual and jitter indicators. Top: XY residual between tracker and detector (RMSE = 0.094 m, p95 = 0.119 m). Bottom: bearing rate magnitude, showing a 55 % jitter reduction from the Kalman filter (detector p95: 137.9 °/s → tracker p95: 62.2 °/s).

Figure 0.16: Kalman filter tracking performance evaluation.

The kalman filter continuous velocity assumption is the key limitation that we could observe. Whenever the human stops or reverses direction, the improved kalman tracker continues predicting forward for several steps before measurements correct it. Adding a decay function helped but didn't fully really fix it.

13 VLA Results in Environment 1 (Table + FR3, Single Tool, Simple Pick-and-Place)

Environment 1 is used as a development environment to integrate both VLA pipelines and validate the training and inference code. TinyVLA was trained up to 10 000 steps on around 50 episodes. While the arm learned the correct motion pattern (approaching the screwdriver then moving to the place pose), the measured metrics stayed at 0% success rate, mainly because gripper close/open supervision is sparse (one transition per episode). Videos of this run are in `tinyvla/env1/` [13].

SmolVLA was trained on 100 episodes for 4 000 steps using the same demonstrations converted to the LeRobot format. This first run already achieved roughly 80 % success rate. Trajectory execution was consistent with the intended pick-and-place cycle, and the policy often attempted a re-approach when the first grasp was missed. Videos are in `smolvla/env1/` [13].

SmolVLA reached usable manipulation behavior faster than TinyVLA in Environment 1, which is coherent with the architecture differences discussed in Part II [21, 17] and the fact that TinyVLA's action head was intialized randomly. The main value of this phase was getting the data collection, training, and deployment pipelines fully operational before moving to the more challenging Environment 2.

14 VLA Results in Environment 2 (Table + FR3, Multi-Object Randomized Scene)

TinyVLA results in Environment 2 TinyVLA was trained on 2000 demonstrations where each demonstration is a successful pick up. The action expert is trained and 5% of VLM weights with LoRa. It's worth noting that in TinyVLA, the action expert is randomly initialized and we're not training on top of a pre-trained action expert. Llava Pythia 700M is used as the VLM backbone. We trained on 5000 steps with a batch size of 16 which took 22.3h to finish on "Nvidia RTX A5000 24gb". The final loss was 0.03 on the evaluation set.

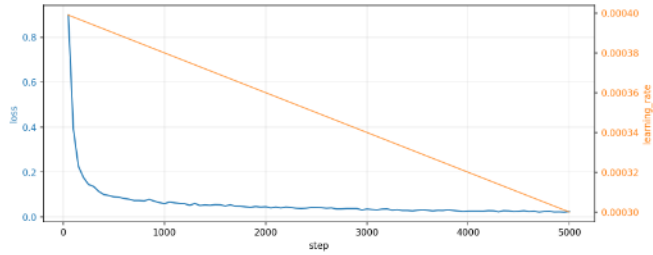


Figure 0.17: TinyVLA training curves over 5000 steps in Environment 2. The blue curve (left axis) shows the loss decreasing rapidly in the first 500 steps before plateauing around 0.03. The orange curve (right axis) shows the learning rate decaying linearly

The success rate was 0% during this run and an average final distance from the object of 7.2cm. The observation is that the model struggles with depth and closes the gripper before reaching the target even when using 3 view cameras. Inference was running at 42% real-time when deployed in IsaacSim.

Cause : The main reason this training run failed with 0% success rate is the fact that the action expert wasn't pre trained on a massive dataset (compared to smolVLA which has an open weights' model trained on 30000 episodes available). We didn't run more experiments with TinyVLA mainly because the training takes very long and is unstable (Matter of fact, using more than 1 data loader causes an "out of memory" error even on 32GB of ram using the trainer available in the TinyVLA repository [21]).

SmolVLA results in Environment 2 Training smolVLA on the same dataset (2000 episodes) for 20000 steps took 12.6h on the same system. The batch size used was 64 with mixed precision to speed up training. The pre-trained model was loaded from `lerobot/smolvla_base` [17].

The evaluation loss has reached 0.002 after 5000 steps of trainig, it stayed consistnet after that , it was the max even though the behavior changed during inference.

At the end of training, we deployed the trained model on IsaacSim in out of distribution environment (Different lighting, different initial end effector pose, different object poses and different cuboid sizes and colors) for 100 episodes using the checkpoints at 5000 steps, 10000 steps, 15000 steps and 20000 steps. The inference runs at 85% real time which is a lot more usable.

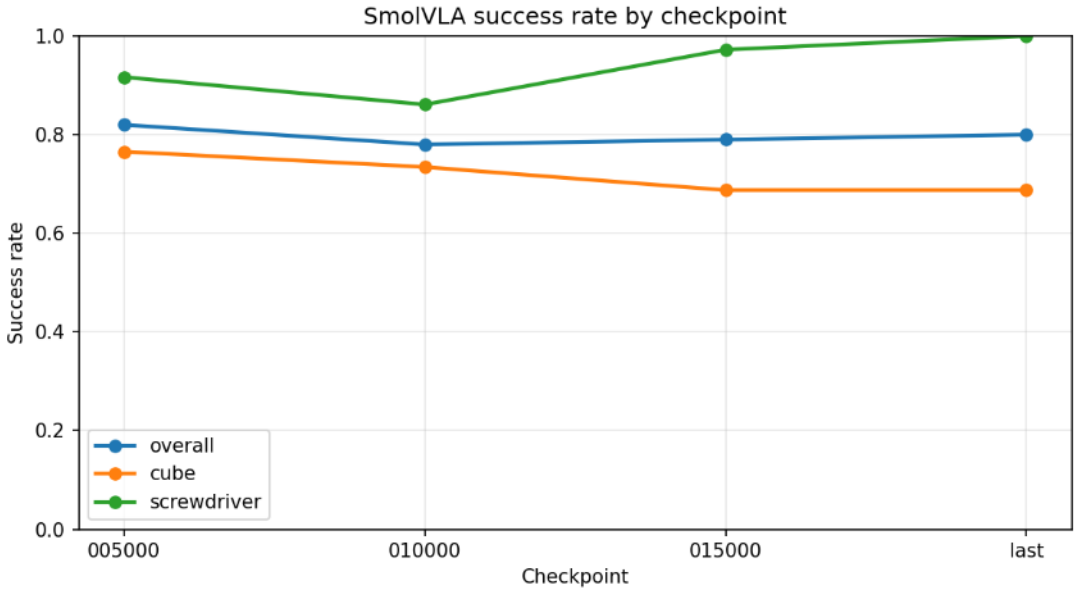


Figure 0.18: SmolVLA success rate by checkpoint in Environment 2. The model achieves an overall success rate around 80%, with perfect screwdriver success rate and near perfect success rate on cuboids

From the figure above, we can see that the pick up success rate stagnated from the first checkpoint at around 80%. Meanwhile, the screwdriver success rate continued improving till reaching 100% success in the final run.

This discrepancy is expected given the difference in visual and geometric variability between the two object categories. All screwdrivers have a similar geometry and even when their size changes, the difference is small. Other than that, only 5 screwdriver models were used during training and the 5 models used during inference are similar visually which makes generalization easy. In contrast cuboids have a far greater domain randomization: the x-dimension ranges from 7 cm to 15 cm, the y-dimension from 0.7 cm to 7 cm, and the z-dimension from 4 cm to 15 cm. Colors are also randomized (12 colors) which means we have a much wider distribution of appearances and grasp geometries. As a result, even with screwdrivers representing only 30% of the training data, the low variance makes them considerably easier to learn than the cuboids.

Environment 2 comparison (TinyVLA vs SmolVLA) From the experiments above, we can see that smolVLA far outperforms TinyVLA on the same dataset with the same conditions even though the VLM backbone is more than 50% larger in TinyVLA. Furthermore, TinyVLA is a lot slower to train and inference which is mainly due to its size but also the efficiency of the model (Layer skipping and visual token reduction in smolVLA) as well as the interleaved cross/self attention in its action expert which is more efficient than the diffusion policy decoder in TinyVLA.

Generalization attempt results

The generalization experiment trained SmolVLA for 60 000 steps on a 10 000-episode dataset and evaluated every 10 000 steps in the warehouse environment and a new 1500 dataset was collected and smolVLA was trained on it to isolate the causes of the failure. The full setup, per-checkpoint success rates, difficulty and object breakdowns, and the subsequent debug run are detailed in Appendix C.

15 VLA Results in Environment 3 (BCR + Mounted Manipulator, Floor Tool to Bin)

Model choice for Environment 3. Based on previous environments, we kept only SmolVLA for Environment 3 and did not continue TinyVLA on this setup.

SmolVLA training/loss .

Figure 0.19: We can see that the training and validation curve does not diverge and keeps going down,it is expected for training loss but for validation it becomes stable and after a certain point stop decreasing in meaningful values, as we will see that the best deployment success is still observed around intermediate checkpoints .

Judging by Figure 0.19, validation loss keeps going down and does not diverge. Around the intermediate checkpoints (about 4,000 to 5,000), train and validation curves start to stabilize. After that point, loss still decreases but only with very small values and the improvement becomes insignificant in practice. However, deployment tests show that best practical checkpoints are consistently around 4,000 to 5,000 steps.

Training demonstrations	Training steps/checkpoints									
	1000	2000	3000	4000	5000	6000	7000	8000	9000	10000
50	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
100	0%	0%	0%	10%	10%	0%	0%	0%	0%	0%
200	0%	0%	15%	60%	50%	25%	15%	0%	0%	0%
400	0%	0%	5%	50%	40%	15%	5%	0%	0%	0%
1000	0%	0%	0%	10%	10%	0%	0%	0%	0%	0%

Table 0.3: Environment 3 SmolVLA checkpoint evaluation using a single metric. Each cell reports average task success rate (%) over 20 deployment episodes for each checkpoint. Success means the policy picks the screwdriver and places it in the bin (for example, 15% means 3 successful episodes out of 20).

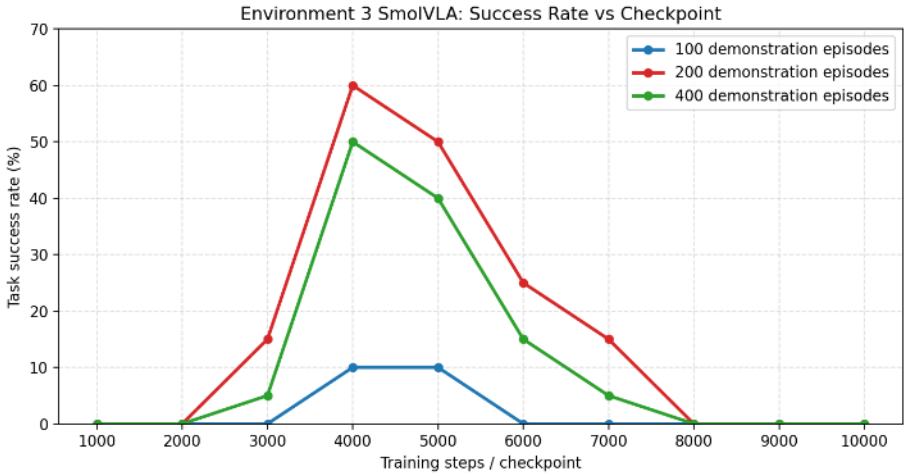


Figure 0.20: Environment 3 SmolVLA task success rate across checkpoints for selected training demonstration budgets (100, 200, 400).

Result discussion. Compared to the previous environments, these results are significantly worse. Environment 1 reached about 80% task success, while the best observed result in Environment 3 is about 60% task success (at 4,000 steps with 200 episodes). This is expected because randomization is harder in Environment 3 and the task is a bit more complicated. In this setup, reaching tools on the floor from a mounted base puts the arm in unusual poses and camera views for VLM-based policies, since most VLA deployments are table-top, which makes adaptation harder.

Results in Environment 3 are also worse than in Environment 2. Even though randomization can be strong in Environment 2, the task there is simpler and faster to execute, while Environment 3 requires a more complex floor pick and bin place sequence. These checkpoint tests also show that using more episodes does not automatically give better performance in this environment, mainly because the scene has high pose variation, floor-grasp geometry is harder for this manipulator, and larger highly varied demonstrations can add noisy action targets. We also suspect we are reaching the practical limits of SmolVLA for this setup, since it is a relatively small model (about 500 million parameters) for this level of variability and task complexity.

Main failure pattern observed. we observed that in our failure runs (by failure we mean the 0% success rate runs) there were two main dominant error types. First one is a small orientation error of the gripper at grasp time , and the second one is a reach error where the gripper closes near the tool but not quite on the corect contact point .So the trained vla policy when deployed can still produce a good behaviour for the trajectory that looks correct globally, while graspping of the tool fails at the contact step.

At the same time, from 3,000 to 10,000 steps, arm behavior is generally good and the policy follows the dataset trajectory structure correctly (approach, grasp attempt, move to bin, return). So for checkpoint selection we keep task success rate as the main metric, because trajectory quality alone is not enough to reflect real task completion.

As we can see in folders `smolvla/env3/success_cases/` and `smolvla/env3/failure_cases/` [13], these videos show concrete examples of the successful and failed behaviors described above.

Part V
Conclusion with Respect to Initial Objectives

16 Conclusion and Objective-by-Objective Assessment

In this project, we created a ROS2 pipeline connecting perception, navigation, human following, manipulation, automated data collection and model training in simulation for an industrial robot manipulator. While the end to end workflow wasn't achieved, the different parts were functional. The mobile part was functional and validated at a usable level with SLAM, nav2, human tracking and the perception layer. The main difficulty was training and achieving acceptable results with VLAs in manipulation. From the comparison between TinyVLA and SmolVLA, we found that TinyVLA did not reach usable manipulation mainly due to the action head being initialized from scratch (No pre trained checkpoint available).

Furthermore, inference was slower because of using DDPM instead of flow matching. SmolVLA on the other hand reached a more usable result at over 80% in the first environment and the second environment. In the harder environment, success rate was 60%. From the generalization attempt and the debug run, we saw that data fedelity mattered substantially, removing the lossy video compression and reducing clutter reached a success rate of 69.2% which means to achieve a good performance, reaching precision, data quality and representation, good domain randomization during training are all equally important. We also found that SmolVLA has a performance ceiling where too much randomization hurts its performance regardless of the number of demonstrations.

As a conclusion, during this project we found out that the small VLA choices selected for this work aren't performant enough for our industrial use case. The achieved success rates are still below what's needed for a real deployment.

Future work

Try bigger VLA models: Our current results suggest that model size is one limiting factor in the hardest randomized grasp scenes. The next step is to test and train bigger and more capable VLA models such as π_0 and $\pi_{0.5}$ with the same environments and metrics used in this report [2, 7]. **End to end chain** Another important step is to connect the different features (after ensuring reliable performance with the VLA) into one full scenario to validate the use case. **Sim to real transfer** In order to deploy the policy in a real robot, we need a sim to real phase with real demonstrations on the real robot as well as richer domain randomization in simulation. **Navigation and visual SLAM** We also plan to evaluate Isaac ROS visual SLAM and compare it with the current 2D lidar SLAM. **Whole body VLA** Another step is to drop using the classical methods for navigation and train a VLA for full body control for the end to end task.

References

[1] Shuanghao Bai et al. *Towards a Unified Understanding of Robot Manipulation: A Comprehensive Survey*. 2025. arXiv: 2510.10903 [cs.RO]. URL: https://arxiv.org/abs/2510.10903.

[2] Kevin Black et al. *π_0: A Vision-Language-Action Flow Model for General Robot Control*. 2026. arXiv: 2410.24164 [cs.LG]. URL: https://arxiv.org/abs/2410.24164.

[3] Anthony Brohan et al. *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*. 2023. arXiv: 2307.15818 [cs.RO]. URL: https://arxiv.org/abs/2307.15818.

[4] Remi Cadene et al. *LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch*. https://github.com/huggingface/lerobot. Hugging Face open-source robotics library. Apache-2.0 license. 2024.

[5] Google. *Google Gemini (Nano Banana) — Image Generation Tool*. Online tool. Image generated with Gemini (Nano Banana). Accessed 2026-02-23. 2026. URL: https://gemini.google.com/.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: https://arxiv.org/abs/2006.11239.

[7] Physical Intelligence et al. *$\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization*. 2025. arXiv: 2504.16054 [cs.LG]. URL: https://arxiv.org/abs/2504.16054.

[8] International Federation of Robotics. *Sales of Service Robots up 30% Worldwide*. IFR press release (World Robotics 2024 Service Robots report). Accessed 2026-02-18. Oct. 2024. URL: https://ifr.org/ifr-press-releases/news/sales-of-service-robots-up-30-worldwide.

[9] Glenn Jocher and Jing Qiu. *Ultralytics YOLO11*. Version 11.0.0. Documentation: https://docs.ultralytics.com/models/yolo11/. License: AGPL-3.0. 2024. URL: https://github.com/ultralytics/ultralytics.

[10] Kento Kawaharazuka et al. *Vision-Language-Action Models for Robotics: A Review Towards Real-World Applications*. Aug. 2025. DOI: 10.36227/techrxiv.175502755.53627529/v1.

[11] Moo Jin Kim et al. *OpenVLA: An Open-Source Vision-Language-Action Model*. 2024. arXiv: 2406.09246 [cs.RO]. URL: https://arxiv.org/abs/2406.09246.

[12] Ilia Larchenko. *VLA Models Overview (YouTube)*. YouTube video. Channel: https://www.youtube.com/@ilialarchenko. Accessed 2026-02-19. 2024. URL: https://www.youtube.com/watch?v=8dZUOo5xWFw.

[13] Gana Lounas and Reda Ould Oulhadj. *Project video drive*. google drive. Contains human-following, tool-detection, and Nav2/SLAM validation runs. Accessed 2026-02-20. 2026. URL: https://drive.google.com/drive/folders/1kwuE1K4L-fm7IALDHTVsm6aSL7GzOoQB?usp=drive_link.

[14] Nav2 Maintainers. *Navigation2 Documentation*. Online documentation. Accessed 2026-02-21. 2026. URL: https://docs.nav2.org/.

[15] NVIDIA et al. *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots*. 2025. arXiv: 2503.14734 [cs.RO]. URL: https://arxiv.org/abs/2503.14734.

[16] Maximo A. Roa et al. "Mobile Manipulation Hackathon: Moving into Real World Applications". In: *IEEE Robotics & Automation Magazine* PP (Mar. 2021), pp. 2–14. DOI: 10.1109/MRA.2021.3061951.

[17] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, et al. “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics”. In: *arXiv preprint arXiv:2506.01844* (2025). URL: https://arxiv.org/abs/2506.01844.

[18] Siemens AG. *The True Cost of Downtime 2024: How much do leading manufacturers lose through inefficient maintenance?* Senseye Predictive Maintenance report. Accessed 2026-02-18. 2024. URL: https://assets.new.siemens.com/siemens/assets/api/uuid:1b43afb5-2d07-47f7-9eb7-893fe7d0bc59/TCOD-2024_original.pdf.

[19] Society for Maintenance & Reliability Professionals (SMRP). *SMRP Solutions, September–October 2016*. Magazine issue PDF. Accessed 2026-02-18. Sept. 2016. URL: https://smrp.org/Portals/0/Solutions%20issues/SMRP%20Solutions%20Sept-Oct%202016.pdf.

[20] SOTAAZ Lab. *Flow Matching vs DDPM: Why ODE Beats SDE in Diffusion Models*. https://blog.sotaaz.com/post/flow-matching-vs-ddpm-en. SOTAAZ Blog. 2025. URL: https://blog.sotaaz.com/post/flow-matching-vs-ddpm-en.

[21] Junjie Wen et al. “TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation”. In: *arXiv preprint arXiv:2409.12514* (2024). URL: https://arxiv.org/abs/2409.12514.

Appendices

Appendix A

Additional Plots, Rollouts, Videos, and Troubleshooting Notes

1 Video Repository Structure and Drive Standard

All videos referenced in Parts III and IV in a google drive [13].

Drive link (clickable): Google Drive Project Video Repository.

```
project_videos/
|-- perception_navigation/
|   |-- yolo_human_following/
|   |-- yolo_tool_detection/
|   `-- nav2_slam_tool_navigation/
|-- data_collection/
|   |-- env1/
|   |-- env2/
|   `-- env3/
|-- tinyvla/
|   |-- env1/
|   |   |-- check_50ep_2k/
|   |   `-- train_10k/
|   `-- env2/
`-- smolvla/
    |-- env1/
    |   `-- train_100ep_4k/
    |-- env2/
    `-- env3/
        |-- success_cases/
        `-- failure_cases/
```

Folders

- `perception_navigation/`: human-following, YOLO detection, and Nav2/SLAM validation videos.
- `data_collection/`: teleoperation/scripted demonstrations per environment.
- `tinyvla/` and `smolvla/`: training/deployment rollouts grouped by environment and experiment stage.

Appendix B

10K Dataset metrics

This dataset is more randomized with a (warehouse scene, FR3, multi-object randomized table with difficulty presets, camera noise and language/visual mismatch). This dataset is the one used for the 60K training run in Appendix C.

Statistics

The target mix is 6961 cubes and 3039 screwdrivers. Average episode length is 258.5 steps.

337 episodes (3.37%) are **language/visual mismatch** episodes where the language instruction asks for an object that is not the scene target. These episodes use a no-pick compliance behavior (the arm does not pick anything).

Dual-object scenes

4406 episodes (44.1%) are dual-object scenes where both a cube and a screwdriver are present on the table and the language instruction chooses the target. Single object episodes are 5594 episodes.

Language/visual mismatch analysis

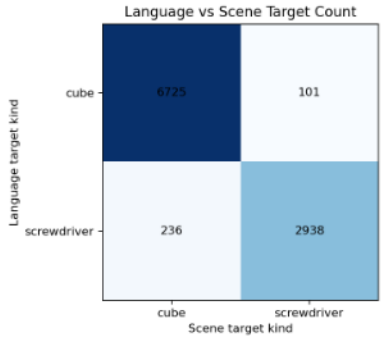


Figure B.1: Language vs scene target confusion matrix. The diagonal (6 725 cube-cube and 2 938 screwdriver-screwdriver) represents matched episodes. Off-diagonal cells ($101 + 236 = 337$) are mismatch episodes where the language asks for the wrong object.

The mismatch rate of 3.37% is intentionally low. The purpose is to teach the policy that when the requested object is not in the scene, it should not attempt a pick. All 337 mismatch episodes have 100% success with no-pick compliance behavior. As noted in the failure hypotheses, this low rate may be too weak to strongly teach language grounding in dual-object scenes during VLA training.

Appendix C

Generalization attempt and debugging

This appendix presents the results of the SmolVLA 60 000-step training run on the 10 000-episode dataset described in Appendix B.

Training setup

SmolVLA was fine tuned from the pre-trained `lerobot/smolvla_base` checkpoint on 10000 episodes for 60000 steps. The optimizer is AdamW with a learning rate of 1×10^{-4}, weight decay 1×10^{-10}, gradient clip norm 10.0 and betas $(0.9, 0.95)$. The scheduler is cosine decay with 1000 warmup steps and 30000 decay steps down to 2.5×10^{-6}. Batch size is 64 with mixed precision. Checkpoints are saved every 5000 steps (12 checkpoints total). Training took around 28 hours to finish.

Evaluation protocol

Each checkpoint at steps 10000 through 60000 (every 10000 steps) was deployed in IsaacSim for 30 episodes in conditions (randomized object poses, sizes, colors, gripper initial pose, warehouse lighting).

Overall success rate

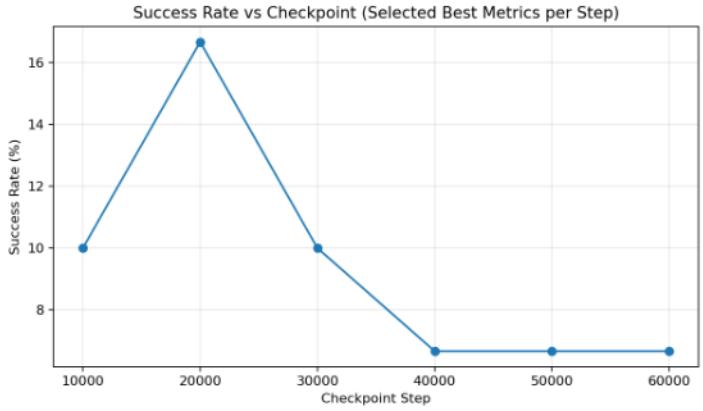


Figure C.1: Success rate vs checkpoint, we have a peak at step 20 000 at 16.7% success rate then decline to a plateau at 6.7% at step 40000

Success rate by difficulty

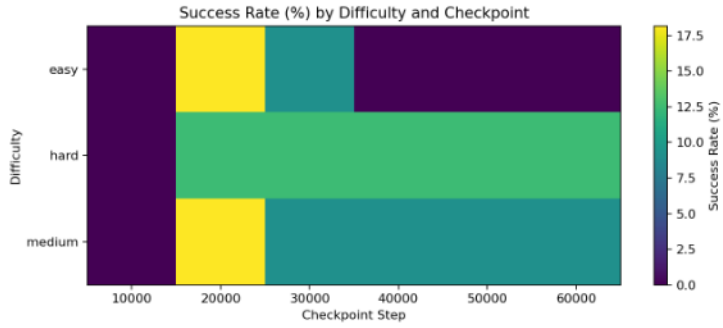


Figure C.2: Heatmap of Success rate by difficulty vs checkpoints. Easy and medium peak at 18.2% at step 20000 then diverge: easy drops to 0% after step 30 000 while medium stabilizes at 9.1%. Hard stays the same at 12.5% from step 20000

As we can see, we have a counter intuitive result where easy episodes drop to 0% success rate after 30000 steps while hard episodes stay at 12.5%. This happened even after inference for 100 episodes. This might be due to the over randomization and video compression which we used to store the episodes in a 256GB disk. (The 2000 episode dataset in the 2nd environment used only lossless compression on png files for around 137gb for the whole dataset while the 10000 episode dataset saved the frames as videos encoded by H264 for a total size of 73GB). While Lerobot's repository supports H264 encoding for the dataset, it's the only explanation we currently have. More tests need to be performed.

Success rate by target kind

Figure C.3: Success rate by target kind across checkpoints. Cube success peaks at 21.7% at step 20 000 then drops to 8.7% from step 40 000 onward. Screwdriver success is 0% at every checkpoint.

The screwdriver 0% success rate across all checkpoints is the least intuitive result. In the 2nd environment (Figure 0.18), screwdrivers reached 100% success with the same model architecture and a smaller 2 000-episode dataset. The difference is the significantly harder environment: the ware-

house scene, camera noise, randomized gripper initial pose and the presence of visual clutter make screwdriver grasping much harder.

Debug run for this environment

A new dataset was collected in the same environment with the only differences being: We no longer use lossy compression and video data. We instead directly use lossless PNG compression and save the data in png format. This creates a significantly larger dataset (Around 80MB per demonstration compared to 8MB per demonstration). The second change is to remove the cluttering (multiple objects on the table). Everything else is kept the same (language mismatch, varying starting position, more randomization in the objects). We collected 1500 episodes.

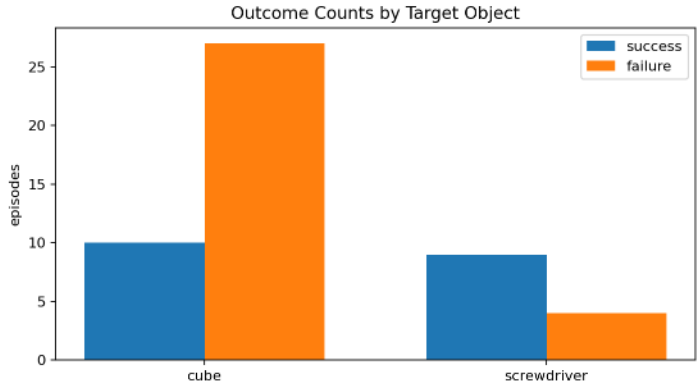


Figure C.4: Outcome counts by target object for the debug run. Cube: 10 successes out of 37 episodes (27.0%). Screwdriver: 9 successes out of 13 episodes (69.2%). Unlike the 60K run, the screwdriver is no longer at 0% success. The problem was mainly the video compression.

Appendix D

State of the Art - Additional Material

This appendix contains detailed state-of-the-art technical formulations moved from Part II.

1 TinyVLA - Detailed Formulation

Efficient VLM backbone and model scales

For the backbone, the authors build compact VLMs (reported range: 70M to 1.4B) and use them as the policy core before robot finetuning [21]. In their experiments they evaluate three TinyVLA scales (Small, Base, Huge). Reported parameter counts are shown below:

Model	Total params	Trainable params	Robot pretraining trajectories
OpenVLA	7.2B	195M	970K
TinyVLA-S	422M	101M	N/A
TinyVLA-B	740M	138M	N/A
TinyVLA-H	1.3B	143M	N/A

Table D.1: Model scale numbers reported in TinyVLA real-world results table [21].

Forward process (noise injection)

Let a_0 be the clean action sample and a_t its noisy version at diffusion step t. Following the standard DDPM formulation [6], the forward process is:

$$q(a_t \mid a_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, a_{t-1},\, \beta_t I\Big),$$

which can be written in closed form as:

$$a_t = \sqrt{\bar{\alpha}_t}\, a_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

Training objective (noise prediction)

The diffusion head learns a network $\epsilon_\theta(a_t, t, c)$ that predicts the injected noise, where c is the TinyVLA conditioning (VLM + state features). The common DDPM simplified objective is [6]:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{a_0,\epsilon,t,c}\Big[\|\epsilon - \epsilon_\theta(a_t, t, c)\|_2^2\Big].$$

So the head learns a denoising direction at each step, conditioned on task context and robot state.

2 SmolVLA - Detailed Formulation

Flow matching objective

The model uses conditional flow matching:

$$x_t = t\epsilon + (1-t)a, \quad u_t = \epsilon - a, \quad \mathcal{L} = \|v_\theta(x_t, o_t, t) - u_t\|_2^2$$

with $\epsilon \sim \mathcal{N}(0, I)$, actions a, and conditioning o_t from VLM features.

Cross and self attention in the expert

The action expert uses an interleaved attention mechanism which alternates between cross and causal self attention layers. It forces the model to ground the predictions in the multimodal features of the VLM backbone + temporal coherence with the predicted action chunk. Cross attention layers use the key value context for action token queries and self attention layers model causal dependencies in action tokens in the action chunks.

During inference, it initializes x_1 as gaussian noise and integrates with a fixed-step Euler update for 10 steps (at each step, the network uses the current noisy guess x_k, the current time step t_k, and the VLM features o_t, then predicts an update velocity v_θ and takes one integration step until the sample becomes an executable action chunk):

$$x_{k+1} = x_k + \Delta t \, v_\theta(x_k, t_k, o_t), \quad \Delta t = -1/K$$

Note: SmolVLA uses AdamW as the optimizer and a cosine decay scheduler.

Fine tuning

Component	behavior	description
VLM vision encoder	Frozen by default	The SigLIP image encoder: turns camera images into visual tokens
VLM language/decoder body	Frozen by default	main LLM part in the VLM
Action expert	Trainable by default	interleaved cross / self attention flow matching transformer
State projection	Trainable by default	linear layer that converts the proprioceptive state to token embeddings
Action projections and time MLPs	Trainable by default	layers that embed noisy action chunk into transformer and flow matching time step

Table D.2: Default fine tuning behavior in SmolVLA [4, 17].

Appendix E

Methodology and Architecture - Additional Material

1 Platforms and Simulation Setup

Robot platform Our final target platform is a mobile manipulator composed of a BCR differential-drive base and a Franka FR3 7-DoF arm with a parallel gripper. We selected this combination because it matches our industrial assistance scenario: long-range movement from the base and dexterous tool interaction from the arm. We chose FR3 also because Franka Research arms are among the most common platforms used to train and validate many early models, so this choice reduced adaptation risk and made comparisons easier. We chose the BCR base because it already provides the full set of components we needed in the right size for our use case, and it is designed for industrial-aisle operation.

Sensors The base side uses front Kinect RGB-D and 2D LiDAR with odometry/IMU streams for localization and following. For manipulation, we use a wrist RGB-D camera (D455 model in our setup) so the grasp policy sees the tool from end-effector viewpoint.

Simulation environments. We used two simulators with different roles:

- Gazebo for mobile behavior: SLAM, Nav2 planning, human-follow tests, and tool-approach navigation in a warehouse world;
- Isaac Sim for manipulation data generation and VLA deployment loops (table setup first, then BCR+FR3 setup).

Why this split. Gazebo gave a fast ROS2 integration loop for navigation/perception blocks, while Isaac gave better rendering and control for generating many pick-place episodes. Because we had one-GPU constraints, we used compact camera resolution and compressed streams for most data collection runs.

Methodologically, we first preferred to validate in a simple standard environment, then once validated we moved to our real and more complex use case.

2 Perception, Tracking, and Navigation Technical Details

Human detection and 3D target extraction

The human-following perception stage uses YOLO11n-pose (Ultralytics, TensorRT deployment) to detect people and keypoints in RGB frames, then recovers one 3D follow target from synchronized RGB-D [9]. We selected the nano variant for real-time operation under compute limits, while keeping acceptable keypoint quality.

Let the set of person detections be $\mathcal{P} = \{(b_i, s_i, \kappa_i)\}_{i=1}^{M}$, where b_i is the box, s_i is confidence, and κ_i are keypoints. The target person index is

$$i^* = \arg\max_i s_i.$$

From the chosen person, the left/right knee pixels (u_L, v_L), (u_R, v_R) are used. Their midpoint is

$$u_c = \frac{u_L + u_R}{2}, \qquad v_c = \frac{v_L + v_R}{2}.$$

Depth is robustified using a local window $W(u_c, v_c)$ (default 5×5):

$$\mathcal{Z} = \{d(p) \mid p \in W(u_c, v_c),\ d(p) > 0,\ d(p) \in \mathbb{R}\}, \qquad z = \text{median}(\mathcal{Z}).$$

With camera intrinsics (f_x, f_y, c_x, c_y), 3D camera coordinates are

$$x = \frac{(u_c - c_x)z}{f_x}, \qquad y = \frac{(v_c - c_y)z}{f_y}, \qquad z = z.$$

Kalman filter for follow-target stabilization

To reduce jitter and survive brief detection dropouts, we track target position and velocity with a linear constant-velocity Kalman filter.

State and measurement:

$$\boldsymbol{x}_k = [x_k,\ y_k,\ z_k,\ v_{x,k},\ v_{y,k},\ v_{z,k}]^\top, \qquad \boldsymbol{z}_k = [x_k,\ y_k,\ z_k]^\top.$$

Transition and observation:

$$\boldsymbol{x}_k = \boldsymbol{A}_k \boldsymbol{x}_{k-1} + \boldsymbol{w}_{k-1}, \quad \boldsymbol{w}_{k-1} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_k),$$

$$\boldsymbol{A}_k = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \boldsymbol{z}_k = \boldsymbol{H}\boldsymbol{x}_k + \boldsymbol{v}_k, \quad \boldsymbol{v}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}),$$

$$\boldsymbol{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

Predict and update:

$$\hat{\boldsymbol{x}}_{k|k-1} = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1|k-1}, \qquad \boldsymbol{P}_{k|k-1} = \boldsymbol{A}_k \boldsymbol{P}_{k-1|k-1} \boldsymbol{A}_k^\top + \boldsymbol{Q}_k,$$

$$\boldsymbol{y}_k = \boldsymbol{z}_k - \boldsymbol{H}\hat{\boldsymbol{x}}_{k|k-1}, \qquad \boldsymbol{S}_k = \boldsymbol{H}\boldsymbol{P}_{k|k-1}\boldsymbol{H}^\top + \boldsymbol{R},$$

$$\boldsymbol{K}_k = \boldsymbol{P}_{k|k-1}\boldsymbol{H}^\top \boldsymbol{S}_k^{-1},$$

$$\hat{\boldsymbol{x}}_{k|k} = \hat{\boldsymbol{x}}_{k|k-1} + \boldsymbol{K}_k \boldsymbol{y}_k,$$

$$\boldsymbol{P}_{k|k} = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H})\boldsymbol{P}_{k|k-1}(\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H})^\top + \boldsymbol{K}_k \boldsymbol{R} \boldsymbol{K}_k^\top.$$

Two practical stabilizers are added:

- **Velocity decay** after prediction:

$$\hat{\boldsymbol{x}}_{k|k-1} \leftarrow \boldsymbol{D}_k \hat{\boldsymbol{x}}_{k|k-1}, \quad \boldsymbol{D}_k = \mathrm{diag}(1, 1, 1, \alpha_k, \alpha_k, \alpha_k), \quad \alpha_k = \exp(-\lambda \Delta t).$$

- **Stationary snap**: if the target remains within radius $r = 0.06$ m for $N = 8$ updates, set

$$\hat{\boldsymbol{x}}_{k|k}(1:3) = \boldsymbol{z}_k, \qquad \hat{\boldsymbol{x}}_{k|k}(4:6) = \boldsymbol{0},$$

  and shrink velocity covariance.

Navigation and following computation details

Navigation is implemented with Nav2 and SLAM Toolbox, using LiDAR + odometry + TF as inputs and `/cmd_vel` as output [14]. The differential-drive base follows the unicycle kinematics:

$$v = \frac{r}{2}(\omega_R + \omega_L), \qquad \omega = \frac{r}{L}(\omega_R - \omega_L).$$

For human following, robot position $\mathbf{p}_r = [x_r, y_r]^\top$ and human position $\mathbf{p}_h = [x_h, y_h]^\top$ define:

$$\mathbf{d} = \mathbf{p}_h - \mathbf{p}_r, \qquad d = \|\mathbf{d}\|,$$

$$s = \min(\max(d - d_{\text{ref}}, 0),\ s_{\max}), \qquad \mathbf{p}_g = \mathbf{p}_r + \frac{s}{d}\mathbf{d}.$$

The goal $\mathbf{p}_g$ is sent through `NavigateToPose`, with heading oriented to the human.

SLAM Toolbox is used as a 2D pose-graph method with scan-matching constraints and loop closures then graph optimization reduces accumulated drift before occupancy-grid generation.

Tool detection and 3D localization details

The tool detector receives synchronized wrist RGB-D frames and returns one semantic tool target for approach. Let $\mathcal{D} = \{(b_i, s_i, c_i)\}$ be detections after thresholding, with box b_i, confidence s_i, class c_i. We select

$$i^* = \arg\max_i s_i.$$

For $b_{i^*} = [x_1, y_1, x_2, y_2]$, the center pixel is

$$u = \frac{x_1 + x_2}{2}, \qquad v = \frac{y_1 + y_2}{2}.$$

Depth robustification on a $k \times k$ window $W(u, v)$ (default $k = 5$):

$$\mathcal{Z} = \{d(p) \mid p \in W(u, v),\ d(p) > 0,\ d(p) \in \mathbb{R}\}, \qquad z = \text{median}(\mathcal{Z}).$$

3D projection with intrinsics:

$$x = \frac{(u - c_x)z}{f_x}, \qquad y = \frac{(v - c_y)z}{f_y}, \qquad z = z.$$

As in the human-perception block, YOLO11-family deployment is selected for real-time operation under the project compute budget [9].

3 Automated data collection

Instead of relying entirely on expert demonstration (data generated through teleoperation with a space mouse or other tools), since we're working in simulation, we decided to automate data collection by creating a scripted policy that generates pick and place or pick only datasets in Isaac Sim in a warehouse environment where the pose of each object is known and the exact pose of the end effector is known. This enables us to have a scalable collection (thousands of randomized synthetic episodes with camera views, proprioception and actions).

Overview

The high level loop of the data collection is:

- **Initialize environment:** We spawn the FR3 robot arm with the table and bin assets or on top of the mobile base, we sample the objects, we add the cameras (3 views) and apply a selected difficulty.
- Control step to update the target pose of the end effector and the gripper command.
- The differential inverse kinematics solver in IsaacSim calculates the joint targets from the EE-pose then the physics is stepped.
- **recording:** The robot state and frames are recorded and added to the episode buffer and success metrics are saved in a .json file.

note: The target is still the mobile manipulator but a part of this data collection environment decouples the robot arm from the mobile base. Since the arm's base will be at a fixed position on the mobile base, the data collected on top of the table will still be useful to train the VLA (a mix of table top episodes and mobile base top episodes). Because all observations are calculated in the arm's base frame, the learned policy would stay invariant to the position in the world (the warehouse or workshop).

Domain Randomization and difficulty

To improve the robustness and generalizability of the manipulation, we added domain randomization organized in 3 difficulty stages.

Difficulty Progression

Starting with an easy preset where we have only a single object on the scene spawned in a rectangle in the workspace: $X \in [0.36, 0.50]$, $Y \in [-0.14, 0.14]$ (base frame). In the medium preset we use a Wider spawning area with moderate dual-object scenes. In the hard preset, we use a wider spawning area and frequent multi objects on the scene creating clutter.

Object Randomization

The initial TCP pose of the robot is randomized per episode. The cuboid sizes and colors are randomized (12 different colors, 32 different sizes randomized at startup). 5 different USD assets with different sizes, shapes and colors. A noise is added to the grasp point to avoid overfitting to a "perfect" grasp point to simulate inaccuracies.

Data logging: We save the dataset in 2 different architectures based on the model we want to train. For TinyVLA, since it uses HDF5 data type, we record the episodes in ROS2 bags containing image data, proprioception and actions which we convert on the fly to HDF5 using a converter script. For SmolVLA, we save the data in a Parquet format used by HuggingFace's Lerobot library. The data is validated at the end for frame consistency and success metrics.

Note about AI usage: During this project, generative ai models and in particular Codex by OpenAI was used to help in code writing especially in IsaacSim scripts where it was the first time using it and in debugging. Nano Banana (image generation model by Google was used to create 2 images in our report (Cited)). In the report, generative AI models were used to fix typos and fix

issues related to Latex compilation. Generative AI models were also used to reformulate some "bad" english phrases.